Ders 8 Genişletilmiş Notu: Strategy, State, Template Method ve Visitor Kalıpları

Ders 8 Genişletilmiş Notu: Strategy, State, Template Method ve Visitor Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Strategy Kalıbı: Amaç ve Motivasyon
- 2. Strategy Kalıbının Yapısı ve Gerçekleme Kararları
- 3. Strategy ile Builder Arasındaki Fark
- 4. Strategy Uygulama Örneği: Kişisel Finans Programında Vergi Hesabı
- 5. State Kalıbı: Amaç ve Yapı
- 6. State ile Strategy Arasındaki Fark
- 7. State Uygulama Örneği: PIM Yazılımında Temiz ve Kirli Durum
- 8. Template Method Kalıbı: Amaç ve Yapı
- 9. Template Method Uygulama Örneği: Farklı Veritabanlarından Kayıt Çekme
- 10. Visitor Kalıbı: Amaç ve Derleyici Örneği
- 11. Visitor Kalıbının Yapısı, Etkileşimleri ve Zayıf Yönleri
- 12. Visitor Uygulama Örneği: Proje Yönetim Yazılımı
- Örnek Sorular
- Dersin Toparlaması

1. Strategy Kalıbı: Amaç ve Motivasyon

Ders, davranışsal (behavioral) kalıplarla devam eder ve ilk olarak Strategy kalıbı işlenir. Hoca, slayttaki genel yapının birçok kalıpta benzer olduğunu, burada önemli olanın kalıbın amacı ve bazı nüanslar olduğunu söyleyerek başlar.

1.1 Amaç

Bir işi yapmanın birden fazla algoritması olabilir. O anda birinci yöntem daha uygunsa onunla, ikinci algoritma daha uygunsa onunla, üçüncüsü daha uygunsa onunla çalışmak isteriz. Yani kullanılan algoritmaya çalışma anında karar vermek ve onu dinamik olarak değiştirebilmek isteriz. Strategy kalıbı temel olarak bu işe yarar.

1.2 Örnek: metin editöründe satır bölme

Bir metin editörü yazdığımızı düşünelim. Metni yazarken ve gösterirken alt satıra geçme (sözcük kaydırma) için birden fazla algoritma vardır. Hoca bunu canlı olarak gösterdi: Word'de iki yana yaslı bir paragrafta bazı boşluklar daha geniş, bazıları daha dar bırakılarak hizalama yapılır. WordPad'de ise "sözcük kaydırma" seçenekleri bulunur:

- Kaydırma yok: hiç alt satıra geçilmez.
- Pencereye kaydır: metin pencereye sığacak şekilde kaydırılır.
- Cetvele kaydır: metin, sayfa kenar boşluğu (marjı) neyse ona uyacak şekilde kaydırılır.

Bir de otomatik heceleme (hyphenation) olabilir: uygun heceden kesme işareti koyup alt satıra geçmek. Hoca, WordPad'de uzun bir kelimenin satıra sığmadığı için satırda çok fazla boşluk kaldığını gösterdi; heceleme yeteneği olsaydı program uygun yere kendisi kesme işareti koyardı. Bunların her biri ayrı bir algoritmadır ve kullanıcı istediğini çalışma anında seçebilmelidir.

1.3 Örnek çözüm (ekrandan, 00:10:00)

Uygulamamız metni düzenlerken sayfaya bölme işini de yapacaktır. İlkemiz "neyin değiştiğini bul ve onu sarmala"dır. Değişen şey satır bölümleme algoritması olduğuna göre önce o sarmalanır.

```
Uygulama {bölmeleyici: SatırBölümleme; metinDüzenle()}
   not: metinDüzenle() içinde  bölmeleyici.bölmele()
Uygulama ◇——► SatırBölümleme {bölmele()}
SatırBölümleme ◁— KesmeliBölümleyici   {bölmele()}
SatırBölümleme ◁— KelimedenBölümleyici {bölmele()}
SatırBölümleme ◁— SınırdanBölümleyici  {bölmele()}
```

`SatırBölümleme` soyut bir üst sınıf olabileceği gibi somut da olabilir (bir varsayılan algoritma, default implementation). Aslında bu kalıp için bir arayüz (interface) olması bile yeterlidir. Ancak algoritma uygulamanın metnine ya da başka verilerine ulaşmak isteyecekse bölmeleyiciden uygulamaya doğru da bir bağlantı olmalıdır; bu durumda soyut üst sınıf kullanmak gerekir. Hoca anlatımı soyut üst sınıf üzerinden sürdürür.

Önemli olan çeşitli somut bölümleyici sınıflar oluşturmak ve her birinin kendi algoritmasına sahip olmasıdır:

- `KesmeliBölümleyici`: ilgili heceden kesme işareti koyup alt satıra geçer.
- `KelimedenBölümleyici`: araya boşluklar koyarak her kelimeyi bir satırda bütün tutmaya çalışır.
- `SınırdanBölümleyici`: başka bir şekilde çalışır.

Hepsi farklı bir algoritmayı aynı `bölmele()` metodu altında gerçekler. Uygulamaya bir `bölmeleyici` üye alanı eklenir. Uygulama metin düzenleme işlerini yaparken bölmeleme işini bu üye üzerinden yaptırır. Kullanıcı seçeneklerden kesmeli bölmeyi seçerse uygulamanın `bölmeleyici` üyesine bir `KesmeliBölümleyici` nesnesi verilir; kelimeden bölmeyi seçerse bir `KelimedenBölümleyici` nesnesi atanır.

2. Strategy Kalıbının Yapısı ve Gerçekleme Kararları

2.1 Genel yapı

Kalıp olarak düşünüldüğünde bir Context vardır. Context'in yapacağı çeşitli işler vardır; bunlardan biri farklı algoritmalarla yapılabilir. O algoritmanın arayüzü soyut bir sınıfta ya da arayüzde, metot imzası ya da imzaları olarak tanımlanır. Alt tarafta çeşitli somut strateji gerçeklemeleri (ConcreteStrategy) yer alır. Context'e hangi somut strateji atanırsa iş onun algoritmasıyla yapılır.

2.2 Tek metot mu, çok metot mu?

Arayüzde tek bir metot olmak zorunda değildir:

- Algoritmanın birden fazla adımı varsa, bu adımların belli bir sırada çalıştırılması gerekiyorsa ve bu sıraya Context'in müdahale etmesi gerekiyorsa birden fazla metot olması yerindedir.
- Context'in tek işi "ilgili veri bu, sen bu işi yap, sonucu döndür" demekse tek bir metot yeterlidir.

Strategy normalde soyut bir sınıftır ve algoritma arayüzü soyut metottur. Varsayılan bir algoritma söz konusuysa bu metot somut da olabilir; alt sınıflar onu ezerek (override) kendi algoritmalarını sağlar.

2.3 Uygun algoritmaya kim karar verir?

Strategy ve Context nesneleri seçilen algoritmayı gerçeklemek için ister istemez etkileşir. Asıl soru, o anda hangi algoritmanın en uygun olduğuna kimin karar vereceğidir:

- **Strateji karar verir:** soyut Strategy sınıfına eklenecek somut (ya da private) bir metotla hangi algoritmanın uygun olduğuna karar verilebilir. Kararın stratejide verilmesi Context'i somut gerçeklemelerden daha bağımsız yapar.
- **Context karar verir:** gerekli analizi yapar, sonucuna göre somut stratejilerden birini ilklendirir ve işi onun üzerinden yaptırır.
- **İstekçi karar verir:** Context'i kullanan başka bir nesne Context'e "şu somut stratejiyle çalış" diyerek somut stratejiyi sağlar.

2.4 Ne zaman kullanılır?

- Bir algoritmanın değişik çeşitlemelerine ihtiyaç duyulduğunda.
- İstekçilerin kullanılacak algoritmaların gerçeklenmesini bilmemesi gerektiğinde. Context yalnız stratejiye "işi yap" der; ayrıntılar somut sınıflardadır.

Slaytta üçüncü madde olarak "yalnız davranışlarında değişiklik gösteren birçok ilişkili sınıf bulunduğunda" yazıyordu. Hoca bu cümlenin kalıba özgü olmadığını, kalıtımın genel yapısına yönelik bir ifade olduğunu söyledi ve maddeyi derste slayttan çıkardı.

2.5 Zayıf yönler

Strategy önemli esneklik sağlayan bir kalıptır, ama birkaç zayıf yönü vardır.

Birincisi, istekçiler değişik stratejilerin varlığından haberdar olmak zorunda kalabilir. Hoca "kalır" değil "kalabilir" dediğinin altını çizdi; birinin bilmesi mecburdur. Context kendisi karar verecekse bütün somut strateji nesneleriyle ilişkide olması gerekir. Ama aynı işe ihtiyaç duyan başka Context sınıfları da olabilir. Örneğin bir Context A ve B stratejileriyle çalışırken C'ye ihtiyaç duymayabilir; başka bir Context C ve D stratejilerine ihtiyaç duyabilir. Karar istekçiye bırakılırsa Context'ler ihtiyaç duymadıkları stratejilerle boş yere eşleştirilmemiş olur. Kalıp ayrıca stratejilerin çalışma anında değiştirilebilmesi esnekliğini de sağlar. Stratejiyi Context'i kullanan istekçi sağlıyorsa algoritma Context'e tamamen şeffaftır.

İkincisi, Context'in sahip olduğu durum bilgisinin tümüne her stratejinin ihtiyacı olmayabilir. Bazı algoritmalar daha çok, bazıları daha az veriye ihtiyaç duyar; ama ortak algoritma arayüzü üzerinden Context bütün veriyi somut stratejiye aktarmak zorundadır. Aynı bellek alanında çalışan nesnelerde bu gerçekten sorun değildir. Ancak nesneler birer servis ise ya da dağıtık bir sistemde ayrı bilgisayarların bellek uzaylarında duruyorsa, belirli bir stratejinin ihtiyaç duymadığı verilerin aktarılması ek yük olarak değerlendirilmelidir.

3. Strategy ile Builder Arasındaki Fark

3.1 Aynı şema (ekrandan, 00:20:00)

Hoca slaytta üstte Builder kalıbının, altta Strategy kalıbının sınıf şemasını yan yana gösterip "Arada fark var mı?" diye sordu:

```
Builder (GoF):
RTFReader {ParseRTF()} ◇──builder──► TextConverter {ConvertCharacter(char),
                                                    ConvertFontChange(Font),
                                                    ConvertParagraph()}
   ParseRTF():  while (t = get the next token) {
                    switch t.Type {
                      CHAR: builder->ConvertCharacter(t.Char)
                      FONT: builder->ConvertFontChange(t.Font)
                      PARA: builder->ConvertParagraph()
                } }
TextConverter ◁── ASCIIConverter   {ConvertCharacter(char), GetASCIIText()}    ┄┄► ASCIIText
TextConverter ◁── TeXConverter     {Convert..., GetTeXText()}                  ┄┄► TeXText
TextConverter ◁── TextWidgetConverter {Convert..., GetTextWidget()}            ┄┄► TextWidget
```

```
Strategy:
Uygulama {bölmeleyici: SatırBölümleme; metinDüzenle()} ◇⟶► SatırBölümleme {bölmele()}
SatırBölümleme ◁⟵ KesmeliBölümleyici, KelimedenBölümleyici, SınırdanBölümleyici
```

Sınıf şeması olarak bakıldığında temel olarak fark yoktur. Hoca `SatırBölümleme`'nin şemada somut çizilmiş olmasına karşın soyut sınıf ya da arayüz olarak düşünülmesi gerektiğini hatırlattı; varsayılan bir algoritma varsa somut da olabilirdi. "Bir tek ufak fark var" dedi; onun dışında yapı aynıdır.

3.2 Fark: yapılan iş ve işin seçilme biçimi

Fark, yapılan işte ve işin seçilme biçimindedir:

- **Builder'da** amaç, karmaşık bir varlığın değişik parçalarını oluştururken hangi parçanın nasıl oluşturulacağına esnek biçimde karar verebilmektir. İstemci builder'a sürekli "şimdi şunu yap, şimdi bunu yap" diye müdahale eder; aynı somut builder'a sık sık metot çağrısı yapar. Builder parçaları nasıl oluşturup birleştireceğini bilir ve denetler.
- **Strategy'de** iş tamamen, ör. kesmeli bölümleyici tarafından yapılır; diğerleri o anda hiç etkin bile değildir. Context strateji gerçeklemesini verir, "al bunu, işi yap" der ve aradan çekilir. En fazla "artık bu algoritma uygun değil" deyip verilerini başka bir somut algoritmaya aktarır. Bu geçişler olabilir ama çok sık olmaz. Üstteki soyut strateji sınıfının, somut algoritmaların işi nasıl yaptığını bilmesine gerek yoktur.

Amaçlar da zaten farklıdır. Hoca, başka kalıplarda da durumun böyle olduğunu söyledi: sınıf şeması açısından yapıları aynı olabilir, fakat "fark ayrıntıda gizlidir".

4. Strategy Uygulama Örneği: Kişisel Finans Programında Vergi Hesabı

4.1 Problem ve switch-case çözümü (ekrandan, 00:23:30)

Uluslararası pazara hitap eden bir kişisel finans programı yazıyoruz. Elde edilen çeşitli gelir kalemlerine karşılık verginin hesaplanması gerekir. Hoca, finans ve maliye alanında çalışanların bileceği gibi ülkelerin vergilendirme kurallarının baştan sona çok farklı olduğunu söyledi; tek ortak nokta, vergilendirilen ya da vergilendirilmeyen çeşitli gelir kalemlerinin olmasıdır. İlk etapta üç pazara hitap edilecek: Türkiye, ABD ve Avrupa Birliği. Ana programın adı `TaskController`'dır. Örneğin kaynağı Design Patterns Explained kitabıdır.

Program, ayarlarından hangi ülkeye göre işlem yapılacağı seçilebilecek şekilde olmalıdır. Bunun için bir ülke kodu alanı (bir enum) ve onun setter metodu eklenir. Program hangi ülke kodunu almışsa ilgili `case`'i işletir; hiçbiri değilse varsayılan bir hesap yapar.

```
«Enum» Country {TR, US, EU}
TaskController {code: Country; + taxAmount(income: double): double; + setCountry(code: Country
TaskController ┈┈► Country
```

```java
package strategy.solution1;
public enum Country { TR, US, EU }
public class TaskController {
    private Country code;
    public void setCountry(Country code){
        this.code = code;
    }
    public double taxAmount(double income){
        switch (code) {
            case TR: return income * 0.18;
            case EU: return income * 0.15;
            case US: return income * 0.12;
            default: return income * 0.10;
} } }
```

Bu kodda ülkeler arasında değişen tek şey vergi oranıdır. Hoca, yalnız bu kod için enum'a gerek olmadığını, bir vergi oranı üye alanının yeterli olacağını kabul etti. Ama çözülecek prob-

lemin çok daha karmaşık olduğunu, her `case`'in tek satırlık bir komut değil içerik olarak çok farklılaşan uzun bir kod olduğunu düşünmemizi istedi; örnek ancak o zaman anlamlıdır.

> **Hocanın vurgusu:** Kendinizi switch-case kullanırken görüyorsanız, bilin ki çok biçimliliğin uygun bir kullanımına ihtiyacınız var demektir.

4.2 Strategy ile çözüm (ekrandan, 00:30:00, 00:26:10)

Kişisel finans programında vergi hesabı tek işlem değildir: gelir planlama, raporlama, sonraki aylar için harcama tahmini gibi özellikler de olacaktır. Raporlama büyük olasılıkla ülkeye göre değişmez, ama vergilendirme değişir. Bu yüzden vergi hesabı stratejiye çevrilir: Türkiye kuralları, Avrupa Birliği kuralları ve ABD kuralları için ayrı sınıflar hazırlanır ve program asıl işi bir vergi kuralı arayüzü üzerinden yaptırır.

```
TaskController {tax: TaxRule; + setTax(tax: TaxRule)} ──► «interface» TaxRule {+ taxAmount(inc
TaxRule ◁┈┈ TRrules, EUrules, USrules   (gerçekleme)
```

```java
package strategy.solution2;
public interface TaxRule {
    public double taxAmount( double income );
}
public class TaskController {
    private TaxRule tax;
    public void setTax(TaxRule tax) { this.tax = tax; }
    public double taxAmount( double income ) {
        return tax.taxAmount( income );
    }
}
public class TRrules implements TaxRule {
    public double taxAmount(double income) {
        return income * 0.18;
    }
}
```

Gerçek dünyada farklı gelir kalemleri parametre olarak verilirdi; ayrıntılara girip konuyu dağıtmamak için toplam kazanç tek parametre olarak verilmiştir. Ayar sonucunda `tax` üyesine ilgili somut vergi kuralı nesnesi atanır ve `TaskController` onun `taxAmount` metodunu çağırarak işi yürütür.

4.3 Kazanç

`taxAmount` metodunun normalde çok uzun ve karmaşık olacağını hatırlayalım. Eski hâlinde her `case` diyelim en az 100 satır içerecekti. Strategy ile 400 satırı aşan tek bir dev sınıf yerine 110-120 satırlık üç ya da dört sınıf olur. Hoca dördüncüyü şöyle açıkladı: switch-case'te bir `default` sayıldığı için bir varsayılan kurallar sınıfı da oluşturmak gerekir (bu varsayılanı da pek güzel bulmadığını söyledi). Sonuçta toplam satır sayısı açısından çok fark olmaz; önemli olan bir işin tek bir yerde yapılmasıdır.

> **Hocanın vurgusu:** İyi tasarımın tanımı: problem hangi düzeyde parçalara ayrılabiliyorsa tasarım da o düzeyde parçalara ayrılabilmelidir. Problem "farklı ülkelerin farklı kuralları" düzeyinde parçalanabiliyorsa, her ülkenin kuralları kendi içinde birer parça olarak tanımlanabilmelidir. Strategy kalıbı bunu sağlar.

4.4 Ülkeye bağlı başka işler eklenirse

Mesele yalnız vergi hesabı değilse, farklı ülkelerin başka finans işlemlerinde de farklılıklar olacaksa ne yapılır? Hoca örnek olarak herkesin bir bireysel emeklilik fonuna zorunlu katkıda bulunmasını ve bu katkının gelire göre değişen bir minimumu olmasını verdi. Bunun hesabı için iki seçenek vardır:

- Yeni bir arayüz (ör. `SavingsRule`) ve onu gerçekleyen ülke kuralları sınıflarıyla ikinci bir strateji kalıbı kurmak.

- Aynı stratejiye yeni bir metot (ör. `calculateMinimumSavings`) eklemek. O zaman arayüzün adını vergi kuralı değil "mali kural" gibi bir şey koymak daha uygun olur.

Hoca birinci yolu tercih eder. Bütün işleri tek bir ağır stratejide toplamak, bir stratejiye en az iki farklı sorumluluk yükler ve uyumu (cohesion) düşürür. Her biri kendi işine odaklanan iki ayrı strateji, sistemin geneline bakıldığında uyumu yükseltir.

4.5 Strategy ve Singleton

Hoca bu örneği çalıştırmadı, çünkü hesaplar karmaşık değildir. Ama bir noktanın altını çizdi: Strategy kalıbı Singleton kalıbıyla birlikte kullanılabilir. Her strateji gerçeklemesi Singleton olmaya adaydır, ama öyle olmak zorunda değildir. Türkiye ile ilgili kurallar sistemin genelinde hep aynıdır; bu değişmeyecek kurallar için birden fazla `TRrules` nesnesi oluşturmaya gerek yoktur. Singleton'da tek bir statik örnek (instance) tutulur, ilklendirilmemişse önce ilklendirilip her istekte o örnek döndürülür. Bu ekleme çözümü pek karmaşıklaştırmaz, ama gerekli de değildir.

Hoca kalıbı şöyle toparladı: Strategy, kalıtımın getirdiği çok biçimlilik esnekliğinden yararlanan, tam olarak onun üzerine kurulmuş bir kalıptır.

5. State Kalıbı: Amaç ve Yapı

5.1 Amaç

State de sınıf şeması açısından Strategy'ye birebir benzeyen bir kalıptır; aradaki nüanslar ayrıca konuşulur.

Amaç şudur: bir nesnenin iç durumu (durum bilgisi, state) değişince davranışının da değişmesini istiyoruz. Bu değişikliğin büyük ölçekli olması istenir: nesnenin önceki çalışma biçimini tamamen değiştirmesi. Bir üye alanın ya da parametrenin değeri değişip farklı bir sonuç döndürmesi kastedilmez; nesne çok farklı işler yapmaya başlar.

5.2 GoF örneği: TCP bağlantısı (ekrandan, 00:34:40)

Bir TCP bağlantısını kodladığımızı düşünelim. Bağlantı belli bir anda üç durumdan birindedir: kurulmuş (Established), dinleme (Listen) ya da kapalı (Closed). Bağlantı hangi durumdaysa çok farklı biçimde çalışır: açık durumdayken tekrar `Open` çağrıldığında farklı, açıkken `Close` geldiğinde farklı, kapalıyken `Close` geldiğinde daha da farklı davranır. `Acknowledge` isteği de açıkken ve kapalıyken farklı işlenir.

Bunu switch-case ile kodlamak mümkündür, ama sakıncaları zaten konuşulmuştur. Önerilen çözüm:

- Neyin değiştiğini bularak sarmalamak: değişen şey TCP bağlantısının durumlarıdır, o hâlde durumlar sarmalanır.
- Parça-bütün ilişkisini kullanmaya teşvik: TCP soketi bir sınıf, soketin durumunu simgeleyen üye ayrı bir nesnedir.

```
TCPConnection {Open(), Close(), Acknowledge()} ◇──state──► TCPState {Open(), Close(), Acknowle
   not: Open() içinde  state->Open()
TCPState ◁── TCPEstablished {Open(), Close(), Acknowledge()}
TCPState ◁── TCPListen      {Open(), Close(), Acknowledge()}
TCPState ◁── TCPClosed      {Open(), Close(), Acknowledge()}
```

Üç ayrı durum sınıfı oluşturulur ve ortak bir durum arayüzünde toplanır. `TCPConnection` her `Open`, `Close`, `Acknowledge` işini, o anda hangi durumdaysa o durum nesnesine yaptırır.

5.3 Durum geçişlerinden kim sorumlu?

Kalıp olarak bakıldığında bir Context ve onun olası durumları vardır. Context'e bir metot çağrısı geldiğinde çağrı o anki duruma iletilir ve Context hangi durumdaysa ona göre farklı çalışır.

Durum değişikliğinden büyük oranda, genellikle Context sorumludur. Geçiş state içine de kodlanabilir, ama o zaman state "şu an bu durumdayım, ne olursa öbür duruma geçeceğim" sorusunun yanıtını ister istemez Context'ten istemek zorunda kalır ve Context'ten çok şey isteyebilir. Durum geçişiyle ilgili bilginin çoğunluğu Context'te olduğuna göre bu karar da Context'te verilmelidir.

5.4 Ne zaman kullanılır? (ekrandan, 00:40:00)

- Bir nesnenin çalışma şeklinin, çalışma anında o nesnenin durumuna göre değişebileceği yerlerde.
- Bir nesnenin durumuna bağlı, karmaşık ve uzun, çok kısımlı karar verme komutlarının verilmesi gerektiği anlarda.

6. State ile Strategy Arasındaki Fark

6.1 Slayttaki karşılaştırma (ekrandan, 00:40:00)

İki kalıbın UML şemaları aynıdır; aradaki nüanslar şunlardır:

- **State:**
  - Context'teki durum değiştikçe Context kendi ConcreteState'ini seçer.
  - State geçişleri açıkça yapılır (Context'te verilen komutlarla ya da ConcreteState'ler içinden). Genellikle Context'te yapılır.
  - Durum geçişleri daha sıktır.
- **Strategy:**
  - Strateji geçişleri seyrektir.
  - Strateji nesneleri Context hakkında daha az şey bilir; yalnız işini yapmaya yetecek kadarını sorgular ve kendilerine gelen parametrelere göre işlem yapar.

Hoca bunu TCP örneğiyle açtı: bir bağlantı o anda hangi durumda olduğunu ve hangi durumdan hangi duruma geçmesi gerektiğini bilir, kendini o duruma geçirir ve işi o durum nesnesine yaptırır. Bağlantının durumu an be an değişebilir. Strateji geçişleri ise daha seyrektir; programın bütün çalışması boyunca hiç değişmeyebilir ya da bir kez değişebilir. Bu, stratejinin kullandığı verinin ne kadar değişken olduğuna ve bu değişkenliğin algoritmayı değiştirme olasılığına bağlıdır; genelde bu olasılık çok yüksek değildir. State nesneleri ise Context'teki durumu daha yakından izler ve daha geniş bir durum bilgisi üzerinden hareket eder.

6.2 Taktik ve strateji benzetmesi

Hoca farkı akılda tutmak için şu benzetmeyi yaptı: strateji uzun vadeli plandır, genel oyun planımızdır. Taktik ise anlık olarak, daha sık değişen ama stratejinin hedeflerine ulaşmak için kullanılan hamlelerdir. Bir maça çıktığımızı düşünelim. Stratejimiz savunma ağırlıklıysa savunma odaklı taktiklerle oynarız ve rakibin durumuna göre taktiğimizi değiştiririz. Koskoca oyun planını değiştirmek ise daha seyrek bir iştir; ör. devre arasında başka bir stratejiye geçilir. Stratejinin daha geniş ve uzun vadeli olduğu düşünülürse iki kalıp arasındaki farkı (sık değişen durum ile seyrek değişen strateji) hatırlamak kolaylaşır.

7. State Uygulama Örneği: PIM Yazılımında Temiz ve Kirli Durum

7.1 Problem (ekrandan, 00:50:00)

Hoca örneği Applied Java Patterns kitabından aldığını, çok beğendiği bir örnek olmadığını ama daha iyisini bulamadığını söyledi.

Bir kişisel bilgi yöneticisi (PIM, Personal Information Manager) yazılımımız var; arkadaşlarımızın iletişim bilgilerini tutuyor. Kayıtlar belli bir anda iki durumdan birinde olabilir: temiz (clean) ya da kirli (dirty). Bir arkadaşımızın iletişim bilgilerinden birini değiştirip henüz veritabanına kaydetmediysek veri kirli durumdadır; kaydettikten sonra temiz duruma geçer.

Hoca bunu Eclipse'te gösterdi: bir Java dosyasında küçük bir değişiklik yapınca dosya adının yanında bir yıldız belirir. Bu, değişikliklerin henüz kaydedilmediğini, dosyanın kirli olduğunu

söyler; kaydedince yıldız kalkar ve dosya temiz duruma geçer. PIM uygulaması da aynı mantıkla çalışır.

Durum kayıt bazında değil, bütün kayıtlar için topluca düşünülür. 100 arkadaşın kaydından biri ya da üçü değiştirilip kaydedilmediyse sayı fark etmez, sistemin geneli kirlidir.

7.2 Sınıf şeması

```
State {+ edit(); + save(f: File, s: Serializable)}
State ◁— CleanState {+ edit(); + save(f: File, s: Serializable)}
State ◁— DirtyState
PIMApplication {+ CLEAN: State; + DIRTY: State; - currentState: State;
                + getCurrentState(): State; + setCurrentState(state: State)}
PIMApplication —► State
PIMApplication —► CleanState, DirtyState
```

Strategy'de istemcinin çeşitli stratejilerin varlığından haberdar olabileceği söylenmişti; State'te bu daha yüksek orandadır. Bu yüzden uygulama hem bir `CLEAN` hem bir `DIRTY` üye alanına sahiptir. Bunlar public olmakla birlikte final ve static'tir, öyle olmalıdır. Uygulamanın şu anki durumu private bir `currentState` üyesinde tutulur ve onun için getter ve setter metotları vardır. Uygulama ilk başta temiz durumda başlar.

7.3 Durumların kodu (ekrandan, 00:48:20)

Durumu değiştirebilecek işlemler `edit` ve `save`'dir. Temiz durumdayken düzenleme yapılırsa kirli duruma geçilmelidir; kaydetmenin ardından da temiz durumda olunmalıdır. Kirli durumda ise pek bir kod yazmak gerekmez (hoca örneği biraz da bu yüzden beğenmediğini söyledi).

Soyut `State` sınıfında `save` işleminin asıl gövdesi kodlanmıştır. Temiz durumdaki kaydetme ile kirli durumdaki kaydetme arasındaki tek fark durum geçişidir; kayıtların veritabanına aktarılması ya da dosyaya yazılması iki durumda da aynıdır. Bu yüzden `save` üst sınıfta tam metot olarak yazılmıştır. `edit` soyut metot olarak bırakılabilirdi, ama hoca onu da üst sınıfa almayı tercih etti.

Hoca derste kodda iki düzeltme yaptı:

- `edit`: asıl düzenleme işi üst sınıfta kodlandığı için `CleanState.edit` önce durumu kirliye çevirmeli, sonra `super.edit()` ile üst sınıfın işini yaptırmalıdır; aksi hâlde düzenleme hiç yapılmamış olur. `edit` soyut bırakılsaydı aynı iş hem `CleanState` hem `DirtyState` içinde iki kez kodlanmak zorunda kalınırdı.
- `save`: kodda önce temiz duruma geçip sonra kaydetmek yazılmıştı. Hoca kaydetme sırasında bir şey ters giderse (dosya yazılamazsa, istisna oluşursa) yanlışlıkla temiz duruma geçilmemesi için sırayı değiştirdi: önce `super.save(f, s)`, sonra temiz duruma geçiş.

Düzeltilmiş slayt kodu:

```java
package dp.state.example;
import java.io.*;
public class CleanState extends State {
    public void edit( ){
        StateFactory.setCurrentState(PIMApplication.DIRTY);
        super.edit();
    }
    public void save(File f, Serializable s) throws IOException {
        super.save(f, s);
        StateFactory.setCurrentState(PIMApplication.CLEAN);
    }
}

public class DirtyState extends State { }
```

> **Ders sonrası not:** Slayttaki kod durum geçişini `StateFactory.setCurrentState(...)` ile yapıyor; sınıf şemasında ise `setCurrentState` metodu `PIMApplication`'da görünüyor. Büyük olasılıkla kodda eski bir sınıf adı kalmıştır. Ayrıca slaytta `DirtyState`'in

üstündeki paket satırı `package dp.flyweight.example;` yazıyor; büyük olasılıkla kopyalama hatasıdır, doğrusu `dp.state.example` olmalıdır.

7.4 İşaretleyici (marker) sınıf

`DirtyState`'in hiçbir kodu yoktur; üst sınıfla tamamen aynıdır, yalnız soyut değil somuttur. Böyle sınıflara işaretleyici sınıf (marker class) denir. Bu iş daha çok arayüzlerle yapılır; bazıları işaretleyici sınıf oluşturmaya pek sıcak bakmaz. Hoca bunun büyük bir sakınca olmadığını, ama yalnız basit bir işaretleyici sınıf çıkmasının bu örneği beğenmemesinin bir nedeni olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Java'da işaretleyici arayüze tanıdık bir örnek `java.io.Serializable`'dır: hiçbir metot içermez, yalnız bir sınıfın nesnelerinin serileştirilebileceğini işaretler. Bu örnekteki `save` metodunun `Serializable` parametresi de bu arayüzdür.

7.5 PIMApplication'da işin duruma yaptırılması

Hoca, uygulamada işin durum nesnesine yaptırıldığının açıkça görünmesi için derste `PIMApplication`'a iki metot ekledi: `edit()` ve `save(File f, Serializable s)`. `edit` çağrıldığında `currentState.edit()`, `save` çağrıldığında `currentState.save(f, s)` çalışır; bunun için `java.io` paketi içe aktarıldı. Hoca önce bir `Record` veri sınıfı (bilgiler, getter ve setter metotlarıyla) oluşturup kayıtları bir listede ya da indeksleme gerekiyorsa bir map yapısında tutmayı düşündü, ama `CleanState.edit` parametresiz olduğu için parametresiz devam etti.

8. Template Method Kalıbı: Amaç ve Yapı

8.1 Amaç ve örnek (ekrandan, 01:10:00)

Template Method'un amacı: bir algoritmanın bazı adımlarını sabit, değiştirilemez biçimde belirleyip bazılarının ayrıntılarını başkalarının belirlemesine izin vermek; başka bir deyişle bu ayrıntıların belirlenmesini ertelemek.

Örnek: yine bir çerçeve program (framework) hazırlıyoruz; farklı belge türleriyle çalışabilecek uygulamalar geliştirmeyi sağlayacak. Çerçeve iki genel sınıf sunar: Belge (`Document`) ve Uygulama (`Application`). Çerçeveyi kullanan programcılar bu sınıflardan kalıtımla kendi sınıflarını türeterek kendi yazılımlarını hazırlar.

Bazı işler belge ve uygulama türünden bağımsız olarak ortaktır. Çerçevenin olabildiğince geniş bir kitleye hitap etmesini isteriz; onu kullanacak programcıların hangi tür belgeler ve uygulamalar hazırlayacağını önceden bilemeyiz, bizim aklımıza gelmeyen şeyler onların aklına gelebilir. Bütün bu çeşitliliğe rağmen bazı işlerin belli bir sırada yapılması kodlanabilir. Bu sıra kritik olabilir: belki çerçevenin kendi gereksinimleri yüzünden bazı adımlar mutlaka yapılmalı, unutulmamalıdır. Madem unutulmaması gerekiyor, unutulmasını engelleyebilir miyiz? Ya da bazı adımların kesinlikle değiştirilmemesi gerekiyorsa, "bunlara dokunmayın, geri kalanında özelleştirme yapabilirsiniz" diyerek hizmetimizi sunmak isteriz. Slayttaki ifadeyle: bir belge açılırken ya da oluşturulurken her türlü ince ayrıntıyı kullanıcıya atmak istemiyoruz; bazı temel adımların uygun sırayla yürütülmesini çerçeve programa bırakmak istiyoruz.

8.2 Örnek çözüm (ekrandan, 01:14:40)

Sınıf şemasında hocanın çizdiği yeşil çizginin üstü çerçevedir, altı çerçeveyi kullanan programcının sınıflarıdır.

```
--- çerçeve ---
Document {Save(), Open(), Close(), DoRead()}         (DoRead soyut)
Application {AddDocument(), OpenDocument(),
             DoCreateDocument(), CanOpenDocument(), AboutToOpenDocument()}   (son üçü soyut)
Application ◇──docs──► Document
--- kullanıcı ---
Document ◁── MyDocument {DoRead()}
Application ◁── MyApplication {DoCreateDocument(), CanOpenDocument(), AboutToOpenDocument()}
```

```
MyApplication ┈┈┈► MyDocument
   not: DoCreateDocument() içinde  return new MyDocument
```

```
void Application::OpenDocument (const char* name) {
    if (!CanOpenDocument(name)) {
        // cannot handle this document
        return;
    }
    Document* doc = DoCreateDocument();
    if (doc) {
        _docs->AddDocument(doc);
        AboutToOpenDocument(doc);
        doc->Open();
        doc->DoRead();
    }
}
```

Bir uygulama farklı türde belgelerle çalışır, ama ne kadar farklı olurlarsa olsunlar kaydetme, açma, kapama ve okuma işlemleri yapılmalıdır. Örneğin kaydederken hangi tür belgeyle uğraşılırsa uğraşılsın belge bir bayt ya da string dizisine çevrilebilir; metin dosyası ile ikili dosya farklı oluşturulur, ama hepsi sonunda bir dosya işaretçisi (file pointer) üzerinden kaydedilir. Açarken de o yolda öyle bir dosya var mı, dosyayı açma yetkisi var mı diye bakılmalıdır. Bunlar dosya türünden bağımsız ortak işlerdir. Dosya işaretçisi elde edildikten sonra dosyanın satır satır ya da bayt bayt nasıl okunacağı ise uygulamanın sorumluluğundadır; çerçeve bunu önceden bilemez. Bu yüzden asıl okuma işi `DoRead` adlı soyut metotla tanımlanır ve somut belge sınıfında gerçeklenir. Türden bağımsız ortak işler ise `Document` soyut sınıfında somut metotlar olarak kodlanır.

Benzer şekilde uygulama, açık belgelerin arasına yenisini eklemeyi (bir dizi, liste ya da map'e eklemek) belgenin türünden bağımsız olarak kendisi yapabilir ve yapmalıdır. Ama bir belgenin oluşturulması için ne gerektiğini bilemez; bunu soyut bir metot imzası (`DoCreateDocument`) olarak tanımlar ve gerçeklemesini somut uygulamaya bırakır. `CanOpenDocument` da öyledir: dosyanın istenen formatta olup olmadığı gibi kararları uygulama verir. Açılabiliyorsa, belge açılmadan önce yapılması gereken ilk işlemler varsa onları yapmak için uygulamaya bir fırsat tanınır: `AboutToOpenDocument` soyut metodu. Bu işlemlerin nasıl yapılacağına somut uygulama (`MyApplication`) karar verir.

`OpenDocument` somut bir metottur. Adımları özetle:

1. Uygulama belgeyi açabilir mi diye bakılır (`CanOpenDocument`, somut uygulamada gerçeklenir). Açamıyorsa devam edilmez.
2. Belge oluşturulur (`DoCreateDocument`, somut uygulamada).
3. Belge açık belgelerin listesine eklenir (çerçevede).
4. Açılmadan önce yapılması gereken ilk işlemler varsa yapılır (`AboutToOpenDocument`).
5. Belge açılır ve okunur (`Open` ortak, `DoRead` somut belgede).

Bu adımların bu sırada yürütülmesi şarttır. Farklı somut belgeler ve uygulamalar için farklı yapılması gereken adımlar için alt sınıflara izin verilir; ama diğer işlemler mutlaka bu sırada yapılmalıdır.

> **Hocanın vurgusu:** `AddDocument` ve `OpenDocument` metotları `final` tanımlanmalıdır ki `MyApplication` bunları ezip kafasına göre davranamasın. Hoca slayttaki C++ sözdiziminde bunun nasıl belirtileceğini bilmediğini, ama Java örneğinde bunun `final` olması gerektiğini söyledi ve derste Java kodundaki ilgili yeri düzeltti.
>
> **Ek bilgi (derste anlatılmadı):** C++11'den beri sanal bir metodu alt sınıflarda ezilemez yapmak için `final` belirteci kullanılabilir (ör. `virtual void OpenDocument(const char*) final;`). Sanal olmayan bir C++ metodu ise zaten çok biçimli olarak ezilmez.

8.3 Kalıbın yapısı

Bir soyut üst sınıf ve ondan türetilmiş somut alt sınıf(lar) vardır. Soyut üst sınıf şablonu, Template Method adlı somut bir metot üzerinden sunar: yapılması gereken adımları yapılması ge-

reken şekilde kodlar, aralarda değişecek ayrıntılar için somut alt sınıfa izin verir. O ayrıntı metotları somut alt sınıfta kodlanır. Slayttaki gerçekleme ayrıntıları:

- Alt sınıflara bırakılacak metotlar Java'da mutlaka `abstract`, C++'ta saf sanal (pure virtual) olmalıdır.
- Üst düzeydeki şablon metot mutlaka `final` olmalıdır.

8.4 Ne zaman kullanılır? (ekrandan, 01:20:00)

- Bir algoritmanın değişmez kısımlarını bir kere kodlayıp değişebilecek kısımlarını alt sınıflar üzerinden gerçeklemek istenildiğinde.
- Alt sınıfların davranışlarının denetlenmesi istenildiğinde.
- Tutarsızlığı ya da kod tekrarını önlemek üzere kardeş sınıfların ortak davranışlarını bir noktada (üst sınıfta) toplamak istenildiğinde.

Hoca üçüncü maddeyi şöyle açtı: kalıtım hiyerarşisinde kardeş sınıflarda benzer işlemler vardır ama tamamen aynı değildir; arada önemli farklı kısımlar vardır. Bu yüzden metot doğrudan üst sınıfta tanımlanamaz, yalnız imzası soyut metot olarak konur ve alt sınıflarda kodlanır. Örneğin beş kardeş sınıftan üçü bir işi benzer şekilde, ikisi farklı yapar; işin farklı alt kalemlerinde ise birinci ve beşinci benzerken diğerleri farklıdır. Benzer ama tamamen aynı olmayan işlemler varsa Template Method kurulur: hepsinde aynı sırayla, aynı şekilde yapılan kısımlar üst sınıfın somut şablon metoduna yazılır; değişen kısımlar soyut metot olarak bırakılır ve kardeşler yalnız bu kısımların somut hâllerini verir.

Hoca bu ihtiyacı kod yazarken pratikte hepimizin hissettiğini söyledi: bir kodu üst sınıfa taşımak isteriz ama taşıyamayız. Uzun uzun düşünseydik bu çözüme belki kendimiz de ulaşırdık; ama artık adını bildiğimiz için böyle bir durumla karşılaşınca doğrudan "Template Method zaten bu işe yarıyor" deyip uygulayabiliriz. Hoca bunu ileride refactoring konusunda da göreceğimizi söyledi.

9. Template Method Uygulama Örneği: Farklı Veritabanlarından Kayıt Çekme

9.1 Problem

Programımız veritabanından kayıtları bir `SELECT` komutuyla çekecek, ama farklı veritabanı yönetim sistemleriyle de çalışabilmeli. Farklı sistemler, bağlantı kurma ve SQL komutlarında çeşitli biçimlendirme farklılıkları demektir. Bütün bu farklara rağmen yapılacak işin değişmeyen temel adımları vardır:

1. Bağlantı komutu (connection string) biçimlendirilir.
2. Bağlantı komutu veritabanına gönderilir.
3. Kayıt seçmek için `SELECT` komutu biçimlendirilir.
4. `SELECT` komutu veritabanına gönderilir.
5. Veritabanı seçilen veri kümesini döndürür ve onun üzerinde işlem yapılır.

Farklı veritabanlarında bağlantı ve seçme komutları farklı biçimlendirilir; ama komutların gönderilmesi ve veri kümesinin dönmesi ortaktır (hoca bunların da değişebileceğini, örnekte ortak kabul edildiğini söyledi). Yani 2, 4 ve 5. adımlar ortak, 1. ve 3. adımlar veritabanına göre değişen adımlardır. Örnek Design Patterns Explained kitabındandır.

9.2 Hangi kalıp?

Hoca burada önemli bir tartışma yaptı. Aslında Strategy de kullanılabilirdi: veri erişimi veritabanının tipine göre farklılaşacağı için uygulamaya bir "veritabanı erişim stratejisi" konur, ör. SQL Server ile çalışılacaksa onun somut stratejisi verilir. Ama State denmez, çünkü burada durum bilgisinin değişmesiyle ilgili bir konu yoktur; farklı erişim biçimleri vardır. Bu soruya State demek yanlış cevap, Strategy demek yakın cevaptır. Gereksinimler yukarıdaki gibi anlatıldığında ise cevap Template Method'dur ve en uygunu budur, çünkü farklı veritabanı sistemleriyle çalışılsa da ortak adımlar vardır.

9.3 Çözüm (ekrandan, 01:25:00)

Uygulama `QueryControl` sınıfıdır ve bir `QueryTemplate` şablonu üzerinden çalışır.

```
QueryControl {- qt: QueryTemplate; + QueryControl(QueryTemplate)}   1 ⟶ 1   «abstract» Query
QueryTemplate {+ doQuery(DBname, querySpec: String) {final}
               ~ formatConnect(DBname: String) {abstract}
               ~ formatSelect(querySpec: String) {abstract}}
QueryTemplate ◁⟵ OracleQT, SQLServerQT, GraphDBQT
```

Slayttaki notlar:

- Komut biçimlendirme metotları iç çalışmayla ilgili görülerek protected tanımlanmıştır.
- Bir sınıf kütüphanesi ya da çerçeve program hazırlanmadığına göre Java için paket görünürlüğü de kullanılabilir (şemadaki ~).

Şablon metot `doQuery`'dir. İçinde beş adım kodlanır: 1. ve 3. adımlar (bağlantı ve seçme komutlarının biçimlendirilmesi) soyut metotlar üzerinden çağrılır ki şablonu gerçekleyen somut veritabanı erişim sınıfları bağlantı ve seçme işlemleri için ince ayarı yapabilsin; 2, 4 ve 5. adımları şablon kendisi yapar. `QueryTemplate` sınıfı soyuttur; `doQuery` ise somut ama `final`dir. Hoca derste ayrıntıları yazmadığını, örnekteki gibi iskelet kod yazdığını söyledi. Somut sınıflarda `formatConnect` ve `formatSelect` somut olarak kodlanır: veritabanına özgü biçimde komutu hazırlayıp bir string döndürür ki üst sınıfın şablon metodu işini yapabilsin. `QueryControl` bir sorgu yapacağı zaman bunu `QueryTemplate` üzerinden yaptırır.

10. Visitor Kalıbı: Amaç ve Derleyici Örneği

10.1 Amaç ve sorun (ekrandan, 01:30:00)

Elimizde bir parça-bütün ilişkisi, ör. bir ağaç yapısı şeklinde, farklı bileşenlerden oluşan bir nesne yapısı vardır.

- **Amaç:** herhangi bir nesne yapısını oluşturan elemanlar üzerinde yapılacak bazı işlemler tanımlamak.
- **Sorun:** nesne yapısı her türden bağıntılar içeren sınıfların örnekleri arasında kurulmuş olabilir. Üzerinde işlem yapılan elemanların sınıf yapılarını ya da kodlarını sürekli değiştirmek istemeyiz. Bu sistemde değişik zamanlarda yeni işlemler tanımlayabilmek de isteriz.
- **Eleştiri:** madem daha yapılacak işlemler var, eylemler ve sorumluluklar tam belirlenmemiş, yani gereksinimler tam belirlenmemiş demektir. Dolayısıyla mevcut sınıf yapılarını değiştirerek farklı bir tasarım yapmak daha doğru olabilir. Hoca bu eleştiride tamamen haklı olunacağını söyledi. Ama zaman baskısıyla işi erken sunmak ve ileride geliştirmeye açıklığı bu yolla sağlamak istediğimiz durumlar olabilir.

10.2 GoF örneği: derleyici

Bir derleyici yazıyoruz. Derleyici kaynak kodu farklı türden token'lara ayırır; bu parçalar (düğüm, node) farklı tiplerdendir. Derleyici bütün parçalar üzerinde çeşitli işlemler yapacaktır: tip denetimi (type checking), kod üretimi (code generation), güzel yazdırma (pretty printing, yani girintileme, renklendirme gibi). Ama farklı tip parçalarda işlemler farklı yürütülür. Ulaşılan düğümlerin kimi bir değişkene referans verme düğümü, kimi bir atama düğümü, kimi bir aritmetik işlem düğümüdür. Aritmetik işlem düğümünde tip denetimi ve kod üretimi yapılabilir, ama güzel yazdırma gerekli görülmeyebilir.

10.3 Ekranda çizilen örnek: `x = y + z`

Hoca `x = y + z` komutunun ayrıştırma ağacını (parse tree) ekranda çizdi:

```
      =            (atama düğümü)
    /   \
   x     +         (x: değişken referansı düğümü, +: aritmetik işlem düğümü)
        / \
       y   z       (değişken referansı düğümleri)
```

İfade derleyicinin belleğinde böyle token'lara ayrılır ve her token bir ağaç düğümüyle gösterilir. Üç farklı düğüm türü vardır: atama, değişken referansı, aritmetik işlem. Bu düğümler üzerinde dolaşılırken:

- Atama (=) düğümünde tip denetimi gerekmez; x düğümünde gerekebilir.
- Toplama düğümünün kendisinde tip denetimi gerekmez, ama çocuk düğümlerinde gerekir: Java için ikisi de tamsayıysa aritmetik toplama yapılır; biri `String` ise + aritmetik toplama yerine string birleştirme operatörü olarak çalışır.

Uzun lafın kısası, derleyici birbiriyle ilişkili varlıkları tek tek dolaşmak (gerekirse ileri geri gitmek) ve her düğümde farklı işlemler yapmak zorundadır.

Her tür düğümde her tür işlemin yapılmasının anlamı yoktur; ör. atama düğümünde güzel yazdırma metodunun olmasına hiç gerek olmayabilir. Ama bir kalıtım ilişkisi kurulduysa boş gövdeli bile olsa o metodu kodlamak gerekir. Hoca bunu Visitor'a yapılabilecek eleştirilerden biri olarak andı. Öte yandan derleyiciye ileride yeni özellikler eklenecekse o işlem gerekebilir, ya da üç işlemden başka dördüncü, beşinci işlemlere ihtiyaç doğabilir.

11. Visitor Kalıbının Yapısı, Etkileşimleri ve Zayıf Yönleri

11.1 Örnek çözüm (ekrandan, 01:40:00)

Derleyici parçaları tek tek ziyaret eder ve her parça üzerinde yalnız gerekli işlemleri yapar.

```
NodeVisitor {VisitAssignment(AssignmentNode); VisitVariableRef(VariableRefNode)}
NodeVisitor ◁── TypeCheckingVisitor  {VisitAssignment(AssignmentNode); VisitVariableRef(Variab
NodeVisitor ◁── CodeGeneratingVisitor{VisitAssignment(AssignmentNode); VisitVariableRef(Variab

Program ◇──► Node {Accept(NodeVisitor)}
Node ◁── AssignmentNode  {Accept(NodeVisitor v)}   not: v->VisitAssignment(this)
Node ◁── VariableRefNode {Accept(NodeVisitor v)}   not: v->VisitVariableRef(this)
```

Yalnız tip denetiminden sorumlu ayrı bir ziyaretçi, kod üretimi için dolaşan ayrı bir ziyaretçi olabilir; bu işlerin aynı anda yapılması gerekmez. Derleyici önce bir `TypeCheckingVisitor` oluşturur; o bütün düğümleri dolaşıp tip denetimini yapar (ilk hatada artık derlemez ama diğer hataları bulmaya devam eder). Hiç hata bulunmazsa aynı ağaç üzerinde bu kez `CodeGeneratingVisitor` dolaşır ve kodu üretir. Böylece sırayla farklı işler için, ama yalnız gerekli işleri yapmak üzere bütün düğümler dolaşılır. Builder'daki gibi "bu düğümü dolaşıyorum, şu işi yapayım; şu düğümü dolaşıyorum, bu işi yapayım" diyen if-else ya da switch-case kullanımından uzaklaşılmış olur.

Ziyaretçi düğümleri nasıl ziyaret eder? Her düğümün bir ziyaretçiyi kabul etme (`Accept`) metodu vardır. Ziyaretçi bir atama düğümünü dolaşıyorsa düğüm ziyaretçiyi kabul eder; ziyaretçi tip denetimi ziyaretçisiyse düğümün bilgilerini okuyarak tip denetimi, kod üretimi ziyaretçisiyse kod üretimi yapar. Böylece herhangi bir ziyaretçi herhangi bir düğüm türü üzerinde iş yapabilir; öbür taraftan her düğüm türü herhangi bir ziyaretçiyi kabul ederek kendi üzerinde, üst arayüzde tanımlı işlemlerin yapılmasına izin verir. Yalnız ilgili düğümde ilgili işler yürütülür, anlamsız işler yürütülmez.

Slayttaki maddeler:

- Düğüm gerçeklemeleri kendi üzerlerindeki gerekli işlemleri çağıracak ziyaretçi gerçeklemelerini kabul eder (`Accept` metodu ile).
- Bu şekilde parçalarda yalnız gerekli işlemler yürütülür.
- Çözümün zayıf yönü: düğümler ziyaretçinin hangi metodunun bu düğüm türü için olduğunu bilmelidir.

11.2 Ek bağlaşım

`AssignmentNode`, kendisinin `VisitAssignment` olarak ziyaret edilmesi gerektiğini kodda söylemek zorundadır. `NodeVisitor` arayüzünde hangi tür düğüm ziyaret edilince hangi metodun çağrılacağı kodlanır; atama düğümü de kendini o şekilde ziyaret ettirebilmek için `VisitAssignment` metodunun tanımlı olduğunu bilmek zorundadır. Bu ister istemez ek bir bağlaşım (coupling) getirir. Hoca en azından ziyaretçinin türüne bağımlılık olmadığını vurguladı: atama

düğümü hangi tür ziyaretçi gelirse gelsin kendini ziyaret ettirir ("ben atama düğümüyüm, beni atama düğümü olarak ziyaret et" anlamında `VisitAssignment`'ı çağırır). Tersi olsaydı daha kötü olurdu. Sonuçta hem düğümlere hem ziyaretçilere ek kod yazmak gerekir. Hoca, gereksinim en başında fazla karmaşık belirlendiği için kurulan altyapının da karmaşık olduğunu, bu karmaşıklık yüzünden Visitor'u pek tercih etmediğini, ama kullanmanın kaçınılmaz olduğu durumlarla karşılaşılabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** `Accept` içinde `v.VisitX(this)` çağrısıyla yürütülecek kodun hem ziyaretçinin hem elemanın somut türüne göre seçilmesine "çift yönlendirme" (double dispatch) denir. Java ve C++ gibi diller metot seçimini çalışma anında yalnız çağrının yapıldığı nesnenin türüne göre yapar (tek yönlendirme); Visitor kalıbı ikinci yönlendirmeyi `Accept` üzerinden elle sağlar.

11.3 Genel yapı

Kalıp yapısında çeşitli elemanlar (Element) vardır ve bunlar bir nesne yapısının (ObjectStructure) parçalarıdır. Her birine soyut bir `Accept` metodu tanımlanır; Element tamamen bir arayüz de olabilir. Somut elemanlar `Accept` metodunda belirli bir ziyaretçiyi kabul etmeyi somut olarak kodlar: "ben ConcreteElementA'yım, beni ConcreteElementA olarak ziyaret etmene izin veriyorum" anlamında ziyaretçinin ilgili metodunu çağırır. Kalıbın güçlü yönü, ziyaretçi arayüzünde elemanlardaki farklı işlemlerin ne olduğunun kodlanmasına gerek kalmamasıdır: "bu elemanı ziyaret ediyorum, onda şu işlemler yapılabilir; öbür tür elemanı ziyaret ediyorum, onda bu işlemler yapılabilir".

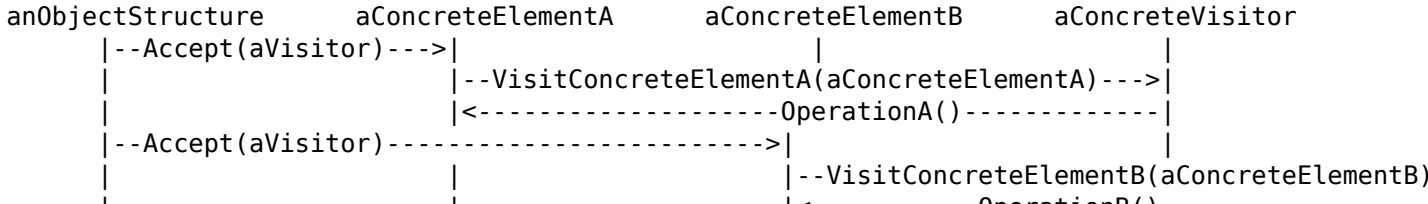
Somut bir ziyaretçi ConcreteElementA'yı ancak A türünde yapılacak işlemler için ziyaret edebilir. Hoca bunun zaten doğal olduğunu ekledi: ConcreteElementA'da `OperationB` diye bir metot tanımlanmazsa olmayan bir metodu kimse çağıramaz.

11.4 Yeni eleman türü eklemenin maliyeti

Kalıbın ciddi bir zayıf yönü yeni somut eleman türleri oluşturmanın zor olmasıdır. Her yeni somut eleman için ziyaretçi arayüzüne ve bütün somut ziyaretçi gerçeklemelerine yeni bir metot eklemek gerekir. Örneğin yeni bir somut eleman çıkarsa hem ziyaretçi arayüzüne onun ziyaret metodu eklenir, hem de daha önce düzgün çalışan ConcreteVisitor1 ve ConcreteVisitor2'nin kodları değiştirilip aynı metot eklenir. İş mantığı açısından değişiklik gerekmese bile derlemeden geçebilmek için bu metotlar eklenmelidir. Belki ConcreteVisitor1'in yeni elemanda gerçekten yapması gereken bir iş vardır; o zaman boş iş yapılmamış olur, ama gerekmeyebilirdi de.

11.5 Kalıp etkileşimleri (ekrandan, 01:50:00)

Hoca kalıbın kolay anlaşılır olmadığını, bu yüzden nesne düzeyindeki işleyişe de bakmak gerektiğini söyledi. Slayttaki dizi şeması:



Nesne yapısı ilk elemana "bu ziyaretçi dolaşacak, sen de bunu kabul et" der. ConcreteElementA ziyaretçiyi kabul eder ve onun `VisitConcreteElementA` metodunu çağırır. A içinde yapılması anlamlı iş `OperationA` ise ziyaretçi elemanın `OperationA` metodunu çağırır; o döner, ziyaret metodu biter, `Accept` biter. Sonra aynı ziyaretçi nesne yapısındaki B elemanını ziyaret eder: B de ziyaretçiyi kabul eder, ama kendisi ConcreteElementB türünden olduğu için `VisitConcreteElementB`'nin çağrılmasını ister; ziyaretçi bu kez `OperationB`'yi çağırır. Varsa bir C elemanı için de aynısı yapılır.

Bu çağrılar iç içedir (nested). Hoca GoF'un daha sade bir dizi şeması gösterimini tercih ettiğini, geri dönüş okları da gösterilecekse kesikli çizgiyle çizilmesi gerektiğini söyleyip dönüş oklarını

ekranda ekledi. Şemanın bile anlaşılmasının kolay olmadığını, ama en azından hangi çağrının hangisinin içinde olduğunu görselleştirdiğini söyledi.

11.6 Ne zaman kullanılır?

- Değişen arayüzlere sahip nesnelerden oluşan bir yapıda (ör. A işlemi, B işlemi gibi), nesneler üzerinde farklı işlemler yapılması gereken anlarda, **VE**
- Bu nesne yapısındaki türlerin değişmediği ya da ender değiştiği koşullarda.

  Hocanın vurgusu: Slayttaki "VE" önemli bir koşuldur. Yeni bir somut eleman türü geldiğinde kalıbı ona uyarlamanın ek işi çok fazladır; oysa tasarım kalıpları değişikliklerin olabildiğince az angaryayla karşılanmasını sağlayan mühendislik çözümleridir.

12. Visitor Uygulama Örneği: Proje Yönetim Yazılımı

12.1 Problem

Bir proje yönetim yazılımı hazırlıyoruz (çok ayrıntılı değil). Temel varlıklar:

- Bir proje hiyerarşisi.
- Projede tanımlı görevler (task). Bazı görevlerin tamamlanması başka görevlere bağlı olabilir: bağımlı görev (dependent task).
- Görev değil proje genelinde düşünüldüğünde projenin ara ya da son çıktıları (deliverable): kod, belge, ürün vb. Hoca bunu illa bilgisayar projesi olarak düşünmemek gerektiğini söyledi.

Örnek Applied Java Patterns kitabındandır. Gün sonunda projenin ne kadar sürede biteceği ve ne kadara mal olacağına ilişkin raporlar oluşturulacak; bizi Visitor'a yönelten kısım budur. Yalnız proje, görev ve ürünleri modellemek için Composite kalıbı ya da yalnız kalıtım hiyerarşisi yeterli olurdu. Ama süre ve maliyeti hesaplamak için bütün görevlerin, alt görevlerin ve ürünlerin tek tek ziyaret edilip kümülatif toplamların alınması gerekir.

12.2 Çözüm 1: kalıpsız (ekrandan, 02:10:00)

Önce nesneye dayalı genel ilkelere uygun, akla gelen ilk mantıklı çözüm yapılır.

```
«Abstract» ProjectItem {- name, description: String;
                        + ProjectItem(String, String); + getName(): String; + getDescription()
                        + setName(String); + setDescription(String)}
ProjectItem ◁── Deliverable {- materialsCost, productionCost: double;
                             + Deliverable(String, String, Contact, double, double);
                             + getOwner(): Contact; + getMaterialsCost(): double; + getProduct
                             + setOwner(Contact); + setMaterialsCost(double); + setProductionC
ProjectItem ◁── Task {- projectItems: ArrayList<ProjectItem>; - timeRequired: double;
                      + Task(String, String, Contact, double); + getProjectItems(): ArrayList<
                      + addProjectItem(ProjectItem); + removeProjectItem(ProjectItem);
                      + getOwner(): Contact; + getTimeRequired(): double;
                      + setOwner(Contact); + setTimeRequired(double)}
ProjectItem ◁── Project {- projectItems: ArrayList<ProjectItem>; + Project(String, String);
                         + getProjectItems(): ArrayList<ProjectItem>;
                         + addProjectItem(ProjectItem); + removeProjectItem(ProjectItem)}
Project ◆──► ProjectItem
Deliverable ──owner 1──► Contact;   Task ──owner 1──► Contact
ProjectCalculator {+ calculateTimeRequired(): double; - calculateTimeRequired(subTask: Task):
                   + calculateCost(): double;         - calculateCost(subTask: Task): double}
ProjectCalculator 1 ──► 1 Project;  Report 1 ──► 1 Project;  Report {+ Report(); + main()}
Report ┈┈► ProjectCalculator;  ProjectCalculator ┈┈► Task, Deliverable
```

Proje, öğelerini bir listede tutar; böylece projeye herhangi bir görev ya da ürün eklenebilir. Her öğenin adı ve tanımı, yapıcısı, getter ve setter metotları vardır. Bağımlı görev şemayı karmaşıklaştırmamak için şemadan çıkarılmıştır (kodda vardır). Bir görev alt görevler ve ürünler

içerebilir, bu yüzden onun da kendi listesi vardır; kalıtım hiyerarşisi bu biçimde modellenince böyle bir yapı kurmak mümkün olur. Görevin alt öğelerini döndüren, öğe ekleyen ve silen metotları vardır. Her görevin bir sahibi (yapacak kişi) vardır; bunun için basit bir `Contact` sınıfı oluşturulmuştur (isim gibi bilgiler, getter ve setter). Görevin ne kadar sürede biteceği `timeRequired` alanındadır.

Ürün için iki ayrı maliyet tanımlanmıştır: malzeme maliyeti ve üretim (hizmet) maliyeti. Bir ürünün de bir sorumlu kişisi olabilir. Hoca, proje sorumlusu da bir kişi olarak düşünülürse `Contact` ilişkisini `ProjectItem`'a çekmenin de mümkün olduğunu, bunun ufak bir ayrıntı olduğunu söyledi. Ürünlerin alt ürünü yoktur; bu da görev ile ürün arasında bir fark yaratır.

Hesaplamaları projeye koymak istemediği için ayrı bir `ProjectCalculator` sınıfı oluşturulmuştur. Ona bir proje atanır, o da bütün öğeler arasında dolaşıp hesap yapar. Kod sınıf şemasından birebir aktarılmıştır. Örneğin `Project.addProjectItem`, öğe listede yoksa ekler (ekrandan, 02:30:00):

```
public ArrayList<ProjectItem> getProjectItems(){ return projectItems; }

public void addProjectItem(ProjectItem element){
    if (!projectItems.contains(element)){
        projectItems.add(element);
    }
}

public void removeProjectItem(ProjectItem element){
    projectItems.remove(element);
}
```

12.3 Süre ve maliyet hesabı

Örnekte ürünün ayrıca bir süresi yoktur; ürün bir görev kapsamında üretilir, görev ne kadar sürüyorsa o kadar sürer. Görevin ise maliyeti yoktur, ürünün maliyeti vardır. Hoca "gerçekte böyle olmuyor" itirazına "o zaman ekleyin" diye cevap verdi; görev ile ürünü olabildiğince birbirinden ayırarak örneği daha rahat uyarlamak istediğini söyledi. Sonuç olarak:

- Süre hesaplanırken görevler arasında dolaşılır; ürünlerde işlem yapılmaz.
- Maliyet hesaplanırken görevlerde doğrudan bir şey toplanmaz, ama bütün ürünlerin ziyaret edilmesi gerekir.

Süre için projenin öğeleri alınır; öğe bir görevse süresi eklenir. Ama görevin alt görevleri olabilir, alt görevin de alt görevi olabilir. Bu yüzden toplam doğrudan yapılmaz; `calculateTimeRequired` metodu `Task` parametreli private bir sürümle aşırı yüklenmiştir (overload). Bu sürüm önce alt görevin kendi süresini alır, sonra onun alt görevleri için kendini özyinelemeli (recursive) çağırır. Böylece hiyerarşi ne kadar derin olursa olsun hesap yapılır. Bu yardımcı metotlar private'tır, çünkü bir alt görevin tek başına süresinin ya da maliyetinin dışarıdan hesaplanması anlamsızdır; bu iç yapıyla ilgilidir.

Maliyet hesabında öğeler dolaşılırken `instanceof` ile türe bakılır (ekrandan, 02:17:40):

```
    for( ProjectItem item : subTask.getProjectItems() ) {
        if( item instanceof Task ) {
            time += calculateTimeRequired((Task)item);
        }
    }
    return time;
}
public double calculateCost( ) {
    double cost = 0.0;
    for( ProjectItem item : project.getProjectItems() ) {
        if( item instanceof Deliverable )
            cost += ((Deliverable)item).getMaterialsCost()+
                    ((Deliverable)item).getProductionCost();
        if( item instanceof Task )
```

```
            cost += calculateCost((Task)item);
    }
    return cost;
}
private double calculateCost( Task subTask ) {
    double cost = 0.0;
    // ... (ekranda görünmüyor; hocanın anlatımına göre aynı mantık alt öğeler için tekrarlanı
}
```

Kodun ilk satırları `calculateTimeRequired(Task subTask)` metodunun son kısmıdır. Öğe bir ürünse malzeme ve üretim maliyeti toplanır. Öğe bir görevse doğrudan maliyeti yoktur, ama alt görevleri ve onların ürünleri olabilir; bunlar private `calculateCost(Task)` ile özyinelemeli olarak dolaşılır. Hoca private sürümün public sürümle neredeyse aynı olduğunu, bunun kopyala-yapıştır kodu (duplicate code) olarak görülebileceğini ve böyle kodda hata yapmanın daha olası olduğunu söyledi.

12.4 Çözüm 1'in sorunu: `instanceof`

Hocanın bu çözümde en çok eleştirdiği kısım her öğeye `instanceof` ile tek tek bakmaktır. Farklı bileşenlerde farklı işlemler yapıldığı için böyle bakmak zorunludur. `instanceof`'tan kurtulmak için `getTimeRequired` ve `getCost` gibi metotları ortak soyut sınıf `ProjectItem`'a eklemek mümkündür, ama bu kalıtımın en doğal kuralını ihlal eder: `getCost` metodunun görev için hiçbir anlamı yoktur, maliyet döndürmesi gerektiği için sıfır döndüren yapay (dummy) bir metot konması gerekir. Aynı şekilde `getTimeRequired`'ın ürün için anlamı yoktur. Kodda sürekli `instanceof` görmek ise çok biçimliliğin doğru kurulamadığını gösterir. Hoca bunu bir yumurta-tavuk durumuna benzetti: ortak metotların hepsini üst sınıfa yazamıyoruz, çünkü alt sınıflarda gerekmeyenler olacak; yazmayınca da türe bakıp ona göre iş yapmak zorunda kalıyoruz.

12.5 Örnek veri ve sonuç (ekrandan, 02:24:00)

Hoca önce hesabın doğru yapıldığını göstermek için `Report` sınıfında örnek bir proje kurdu: lisansüstü eğitim. Bir `Task`'ın yapıcısı ad, açıklama, sahip ve gerekli süreyi alır; süreler hafta cinsindendir. Bir `Deliverable`'ın yapıcısı ad, açıklama, sahip, malzeme maliyeti ve üretim maliyetini alır; maliyet harcanan emek olarak saat cinsindendir. Sahip alanları `null` bırakılmıştır.

```
package dp.visitor.solution1;

public class Report {
    private Project project;
    public Report() {
        project = new Project("Yüksek Lisans", "Lisansüstü eğitim (Yüksek lisans)");
        Task gorev; Deliverable urun;
        gorev = new Task("2014-1 dersleri","2014-2015 Güz yarıyılı (4 ders)",null,15);
        urun = new Deliverable("BLM 5128", "Yazılım Kalitesi ve Test Teknikleri", null, 45, 30
        gorev.addProjectItem(urun);
        urun = new Deliverable("BLM 5130", "Nesneye Dayalı Tasarım ve Modelleme", null, 45, 40
        gorev.addProjectItem(urun);
        urun = new Deliverable("BLM 5116", "İleri Algoritma Analizi ve Tasarımı", null, 45, 35
        gorev.addProjectItem(urun);
        urun = new Deliverable("BLM 5140", "Veri Madenciliği ve Bilgi Keşfi", null, 45, 40);
        gorev.addProjectItem(urun);
        project.addProjectItem(gorev);
        gorev = new Task("2014-2 dersleri","2014-2015 Bahar yarıyılı (4 ders)",null,15);
        urun = new Deliverable("BLM 5126", "İleri Programlama Dilleri", null, 45, 30);
        gorev.addProjectItem(urun);
        // ... (devamı ekranda görünmüyor)
```

Her dönemin dersleri 15 haftalık bir görevdir; her ders bir üründür ("dersten alınan geçer not ürünüdür"). Her ders haftada 3 saatten 15 haftada 45 saat derse girmeyi gerektirir (malzeme maliyeti), ayrıca sınavlar için ek çalışma saati vardır (üretim maliyeti): ilk ders için 30 saat, hocanın "biraz daha zor" dediği ders için 40 saat, diğerleri için 35 ve 40 saat. İki dönem,

her dönemde dört ders vardır. Bunların yanında derslerle aynı anda yürütülebilen bir çalışma görevi ve tezle ilgili bir hazırlama ve sunma görevi de eklenmiştir.

Programı çalıştırınca çıktı şudur (ekrandan, 02:30:00):

```
Proje adı ve açıklaması: Yüksek Lisans: Lisansüstü eğitim (Yüksek lisans)
Proje süresi: 60.0 hafta.
Proje maliyeti: 1174.0 saat çalışma.
```

Hoca 60 haftayı şöyle açıkladı: birinci ve ikinci dönem derslerle (15'er hafta), üçüncü ve dördüncü dönemler teze odaklanarak geçer. Tahminlere göre 1174 saat çalışma gerekir. Ekranda görünen ilk dönemin ürün maliyetleri $75 + 85 + 80 + 85 = 325$ saattir; toplamın geri kalanı ekranda görünmeyen kalemlerden gelir.

12.6 Çözüm 2: Visitor ile

Önceki çözümle aynı kalan kısımlar yeni şemada tekrarlanmamıştır. Değişenler:

- Bir `ProjectVisitor` arayüzü vardır. Maliyeti hesaplayan ayrı bir `ProjectCostVisitor`, zamanı hesaplayan ayrı bir `ProjectTimeVisitor` vardır.
- Rapor önce bir maliyet ziyaretçisi oluşturup proje üzerinde dolaştırır; bütün proje düğümleri ziyaret edilir ve maliyet bulunur. Sonra bir zaman ziyaretçisi oluşturup aynı şekilde toplam süreyi bulur ve ikisini ekrana yazar.
- Ziyaretçi bağımlı görevleri dolaşırken (bu örnekte bağımlı görev görevle aynı işlenir) ürünleri dolaşırken farklı çalışır; ürünler yalnız maliyet hesabı için anlamlıdır. Projenin geneli de ziyarete açık olmalıdır ki bütün bileşenler dolaşılabilsin.
- `ProjectItem`'a soyut bir `accept` metodu eklenir. `Task.accept` `visitTask`'ı, `Deliverable.accept` `visitDeliverable`'ı, `Project.accept` `visitProject`'i çağırır.

Hoca Eclipse'in dosya karşılaştırma özelliğiyle iki çözümdeki `Project`, `Task` ve `Deliverable` sınıflarını karşılaştırdı: iç mantıkta hiçbir şey değişmemiş, yalnız `accept` metodu eklenmiştir. `Project` için (ekrandan, 02:30:00):

```
public void accept(ProjectVisitor v){
    v.visitProject(this);
}
```

Asıl değişen raporlama kısmıdır. `ProjectVisitor` arayüzünde `visitDependentTask`, `visitDeliverable`, `visitTask` ve `visitProject` metotları vardır. Ziyaretçi, projeyi alır almaz dolaşmaya başlar: yapıcısında projeden kendisini kabul etmesini ister (ekrandan, 02:40:00):

```
package dp.visitor.solution2;

public class ProjectCostVisitor implements ProjectVisitor {
    private double cost = 0.0;

    public ProjectCostVisitor(Project project) {
        project.accept(this);
    }

    public void visitDependentTask(DependentTask p) {
        visitTask(p);
    }

    public void visitDeliverable(Deliverable p) {
        cost += p.getMaterialsCost()+p.getProductionCost();
    }

    public void visitTask(Task p) {
        for( ProjectItem item : p.getProjectItems() ) {
        // ... (devamı ekranda görünmüyor)
```

Maliyet ziyaretçisi bir ürünü ziyaret ederken malzeme ve üretim maliyetlerini kümülatif toplama ekler. Bir görevi ziyaret ederken toplama bir şey eklemez, ama alt öğelerini dolaşır ki

gerekirse alt ürünleri de ziyaret edilsin ve maliyet birikmeye devam etsin. Sonunda toplamı döndüren bir getter ile hesaplanan maliyet alınır.

Zaman ziyaretçisi (ekrandan, 02:33:30):

```
public ProjectTimeVisitor(Project project) {
    project.accept(this);
}

public void visitDependentTask(DependentTask p) {
    visitTask(p);
}

public void visitDeliverable(Deliverable p) {
}

public void visitTask(Task p) {
    time += p.getTimeRequired();
    for( ProjectItem item : p.getProjectItems() ) {
        item.accept(this);
    }
}

public void visitProject(Project p) {
    for( ProjectItem item : p.getProjectItems() ) {
        // ... (hocanın anlatımına göre: item.accept(this);)
```

`visitDeliverable` içinde bir şey yapılmaz, çünkü ürünlerde süre tutulmaz. Bir görev ziyaret edildiğinde, bu bir zaman ziyaretçisi olduğu için görevin süresi alınır, sonra bütün alt öğelerinden ziyaretçiyi kabul etmeleri istenir. Proje için ilk toplama gerek yoktur; projenin bütün öğelerinden ziyaretçiyi kabul etmeleri istenir. Öğe bir görevse `accept` üzerinden `visitTask`, ürünse `visitDeliverable` çalışır (zaman ziyaretçisinde ürün için bir şey yapılmaz, iş mantığı öyle kurulmuştur).

`Report`'ta projeyi oluşturan kısım tamamen aynıdır. Değişen tek şey sondadır: önce bir maliyet ziyaretçisi oluşturulup projeye verilir, adı, açıklaması ve maliyeti ondan alınıp yazdırılır; sonra benzer şekilde bir zaman ziyaretçisi oluşturulup toplam süre alınır. İki raporu karşılaştırınca satırlar ve değerler aynıdır, yalnız `main` kısmı biraz farklıdır. Çalıştırınca Visitor çözümü de 60 hafta ve 1174 saat sonucunu verdi. Böylece kalıp hem uygulama mantığı açısından doğru kodlanmış, hem de sınıf şeması kalıbın bütün gereksinimlerini karşılamış oldu.

12.7 Ne kazandık, ne kaybettik?

Kazanç: eski `ProjectCalculator`'daki `instanceof` denetimlerinden kurtulundu.

> **Hocanın vurgusu:** Hoca `instanceof`'a "gerekli kötülük" (necessary evil) olarak bakar: bazen gerekir, ama mümkün olduğunca kullanımından kaçınmak gerekir.

Kayıp: `ProjectTimeVisitor` içindeki `visitDeliverable` metodunun içi boştur; boş gövdeli metotlar da bir rahatsızlık konusudur. Hoca bunun, kalıtımda bazı metotların ya da üyelerin "reddi miras" yapılamamasından kaynaklandığını, genel olarak nesneye dayalı kalıtım mekanizmasında reddi miras olmamasının doğru bir karar olduğunu söyledi. Yani `instanceof` gitti ama bu durum ortaya çıktı; en azından yalnız bir boş metot çıktı. Zaman açısından görev ziyaretinde bile en az bir satırlık iş (alt görevlerin ziyaret ettirilmesi) vardır.

Ek maliyet: iki çözüm arasında kod farkı çok değildir, yalnız angaryası az, ufak bir altyapı eklenmiştir. Bu altyapı sayesinde projeyle ilgili zaman ve maliyet dışında başka analizler yapmak kolaylaşır: yeni bir raporlama türü gerektiğinde ona bir ziyaretçi eklenir, iş çözülür; üst taraftaki sınıflara hiç dokunulmaz. Ama yapıya yeni bir düğüm türü eklemek gerekirse çok iş yapılmış olur. Hoca dürüstçe ekledi: önceki çözümde de yeni bir hesap için yalnız `ProjectCalculator`'a dokunulurdu, bu açıdan da çok büyük bir avantaj yoktur.

Sonuç olarak Visitor çok fazla bir şey kazandırmadı. Ama mükemmeliyetçi yaklaşılırsa nesneye dayalı ilkeleri daha az ihlal eden bir çözüme ulaşıldı ve ek maliyet pek fazla değil; hoca bu

yüzden yapmaya değer olduğunu düşünüyor. Öte yandan “accept’ler, visit’ler, ne neyi kabul etti, ne neyi ziyaret etti, fazla karışık” deniyorsa, ya da ekibe katılacak deneyimsiz ya da bu kalıbı hiç duymamış bir programcının şirketin kod havuzuna uyum sağlaması zor olacaksa, Visitor’lu çözümü tercih etmemek de haklı bir karardır. Tercih tasarımcınındır.

Örnek Sorular

Soru 1 (ekrandan, 00:20:00)

Slaytta üstte Builder kalıbının (RTFReader, TextConverter ve ASCIIConverter, TeXConverter, TextWidgetConverter), altta Strategy kalıbının (Uygulama, SatırBölümleme ve üç somut bölümleyici) sınıf şeması gösterilip soruluyor: “Arada fark var mı?”

> **Cevap (derste verildi):** Sınıf şeması olarak temelde fark yoktur. Fark yapılan işte ve işin seçilme biçimindedir. Builder’da amaç karmaşık bir nesnenin parçalarının nasıl oluşturulacağına esnek biçimde karar vermektir; istemci aynı somut builder’a sürekli adım adım çağrı yapar ve builder parçaları nasıl birleştireceğini bilir ve denetler. Strategy’de Context somut stratejiyi bir kez verip aradan çekilir; belli bir anda yalnız bir strateji etkindir, geçişler seyrektir ve soyut strateji somut algoritmaların ayrıntısını bilmez.

Soru 2 (derste sözlü, 01:22:56)

Programımız farklı veritabanı yönetim sistemlerinden `SELECT` ile kayıt çekecek. Adımlar: bağlantı komutunu biçimlendir, gönder; seçme komutunu biçimlendir, gönder; dönen veri kümesini işle. Biçimlendirme adımları veritabanına göre değişiyor, diğerleri ortak. Hangi kalıp kullanılmalıdır: State, Strategy, Template Method?

> **Cevap (derste verildi):** State yanlış bir cevaptır, çünkü durum bilgisinin değişmesiyle ilgili bir konu yoktur, yalnız farklı erişim biçimleri vardır. Strategy yakın bir cevaptır: uygulamaya bir veritabanı erişim stratejisi konabilir. En uygun cevap Template Method’dur, çünkü ortak adımlar vardır: `QueryTemplate` soyut sınıfında `final` bir `doQuery` şablon metodu beş adımı sırayla yürütür; değişen 1. ve 3. adımlar soyut `formatConnect` ve `formatSelect` metotlarıyla `OracleQT`, `SQLServerQT`, `GraphDBQT` gibi alt sınıflara bırakılır.

Soru 3 (ekrandan, 00:23:30)

Vergi hesabının switch-case’li çözümünde (`TR`: 0.18, `EU`: 0.15, `US`: 0.12, `default`: 0.10) hangi tasarım sorunu vardır ve Strategy ile nasıl çözülür?

> **Cevap (derste verildi):** Her `case` gerçekte ülkeye özgü, yüzlerce satırlık karmaşık bir hesaptır ve hepsi `TaskController` içinde tek bir dev sınıfta toplanır. Kendini switch-case kullanırken bulmak, çok biçimliliğe ihtiyaç olduğunun işaretidir. Strategy ile `TaxRule` arayüzü (`taxAmount(double income)`) tanımlanır; `TRrules`, `EUrules`, `USrules` (ve bir varsayılan kural sınıfı) bu arayüzü gerçekler; `TaskController` bir `TaxRule` üyesi tutar ve `setTax` ile atanan kurala `tax.taxAmount(income)` diye işi yaptırır. Toplam satır sayısı pek değişmez, ama her ülkenin kuralı tek bir yerde durur. Somut kurallar Singleton olmaya da adaydır.

Soru 4 (ekrandan, 01:50:00)

Visitor kalıbı hangi koşullarda kullanılmalıdır ve yapıya yeni bir somut eleman türü eklemek neden pahalıdır?

> **Cevap (derste verildi):** Değişen arayüzlere sahip nesnelerden oluşan bir yapıda nesneler üzerinde farklı işlemler yapılması gerektiğinde **ve** bu yapıdaki türler değişmediğinde ya da ender değiştiğinde. Yeni bir somut eleman eklenince ziyaretçi arayüzüne onun için yeni bir `visit` metodu eklenmeli ve bütün somut ziyaretçiler, iş mantığı gerektirmese bile derlemeden geçebilmek için bu metodu gerçeklemelidir.

Dersin Toparlaması

Bu derste dört davranışsal kalıp işlendi. Strategy, bir algoritmayı nesne olarak sarmalayıp Context'e atayarak çalışma anında değiştirilebilir kılar; kalıtımın çok biçimliliği üzerine kuruludur. Vergi örneğinde switch-case yerine ülke başına bir kural sınıfı kullanıldı. State, Strategy ile aynı şemaya sahiptir. Ama Context kendi durumunu bilir, durumlar arasında kendisi ve sık geçer, durum nesneleri Context'i daha yakından izler; PIM örneğinde temiz ve kirli durumlar bunu gösterdi. Template Method, bir algoritmanın değişmez adımlarını üst sınıftaki `final` bir şablon metotta sabitler, değişen adımları soyut metotlarla alt sınıflara bırakır; belge çerçevesi ve veritabanı sorgusu örnekleriyle anlatıldı. Visitor, bir nesne yapısının elemanları üzerinde yeni işlemleri elemanların sınıflarını değiştirmeden eklemeyi sağlar. Bedeli ek bağlaşım, boş gövdeli metotlar ve yeni eleman türü eklemenin zorluğudur; proje yönetimi örneğinde `instanceof`'lu çözüm ile Visitor'lu çözüm aynı sonucu (60 hafta, 1174 saat) verdi.

Dersin ortak mesajı, sınıf şemaları aynı olan kalıpların amaç ve ayrıntıda ayrıldığı ve bir problemi doğru kalıpla eşleştirmek için gereksinimlerin dikkatle okunması gerektiğidir. Davranışsal kalıplardan geriye Mediator kaldı; hoca bir sonraki derste Mediator'ı kısaca tamamlayıp kod kokuları (code smells) ve yeniden düzenleme (refactoring) konusuna geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.